

DART: Distributed Assignment of Research Tasks for Heterogeneous Compute Environments



Abdur Rouf, Fahad Ahmad Khan, Benjamin Keene, Shafaq Chaudhry and Murat Yuksel

Networking and Wireless Systems Lab (NWSL), Department of Computer Engineering, University of Central Florida

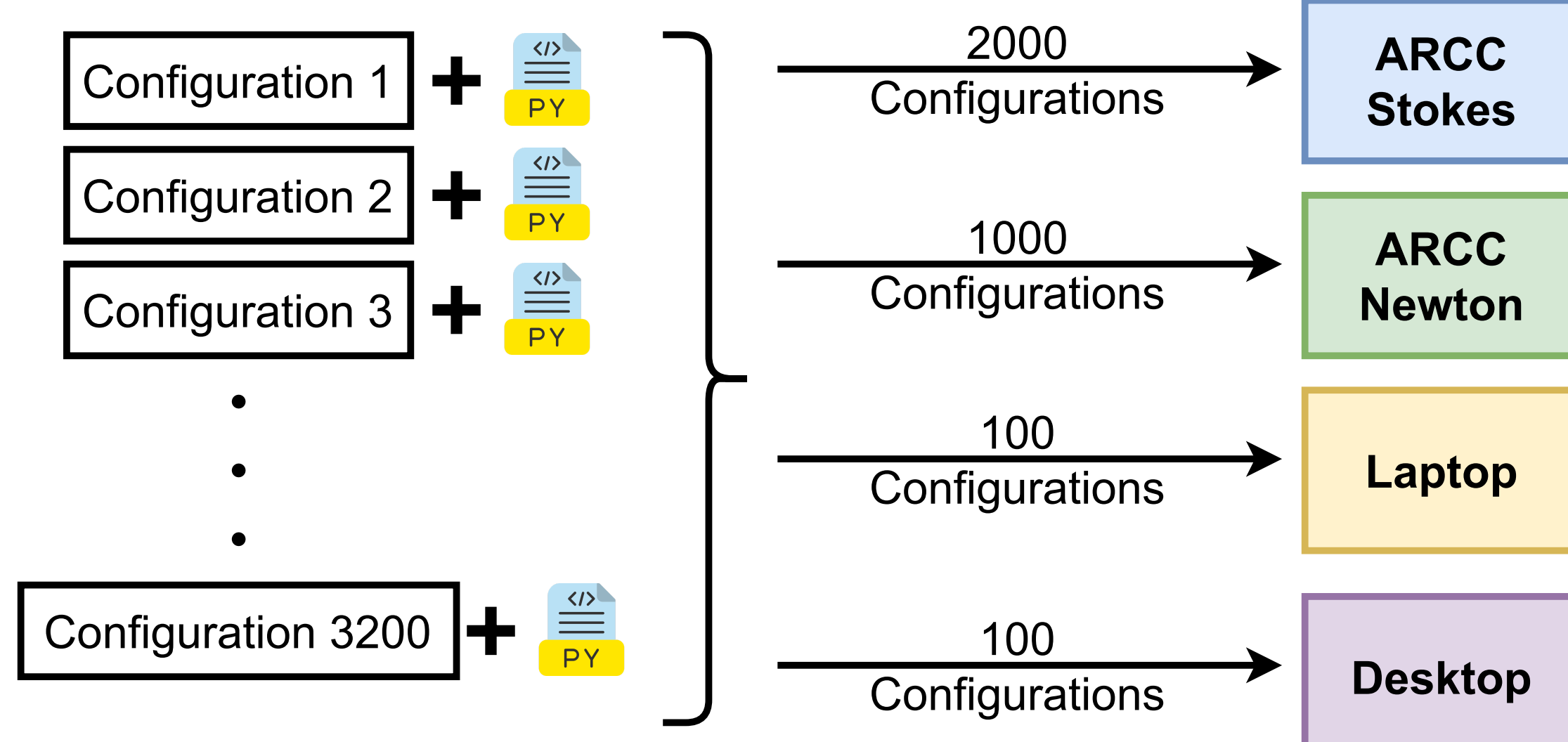
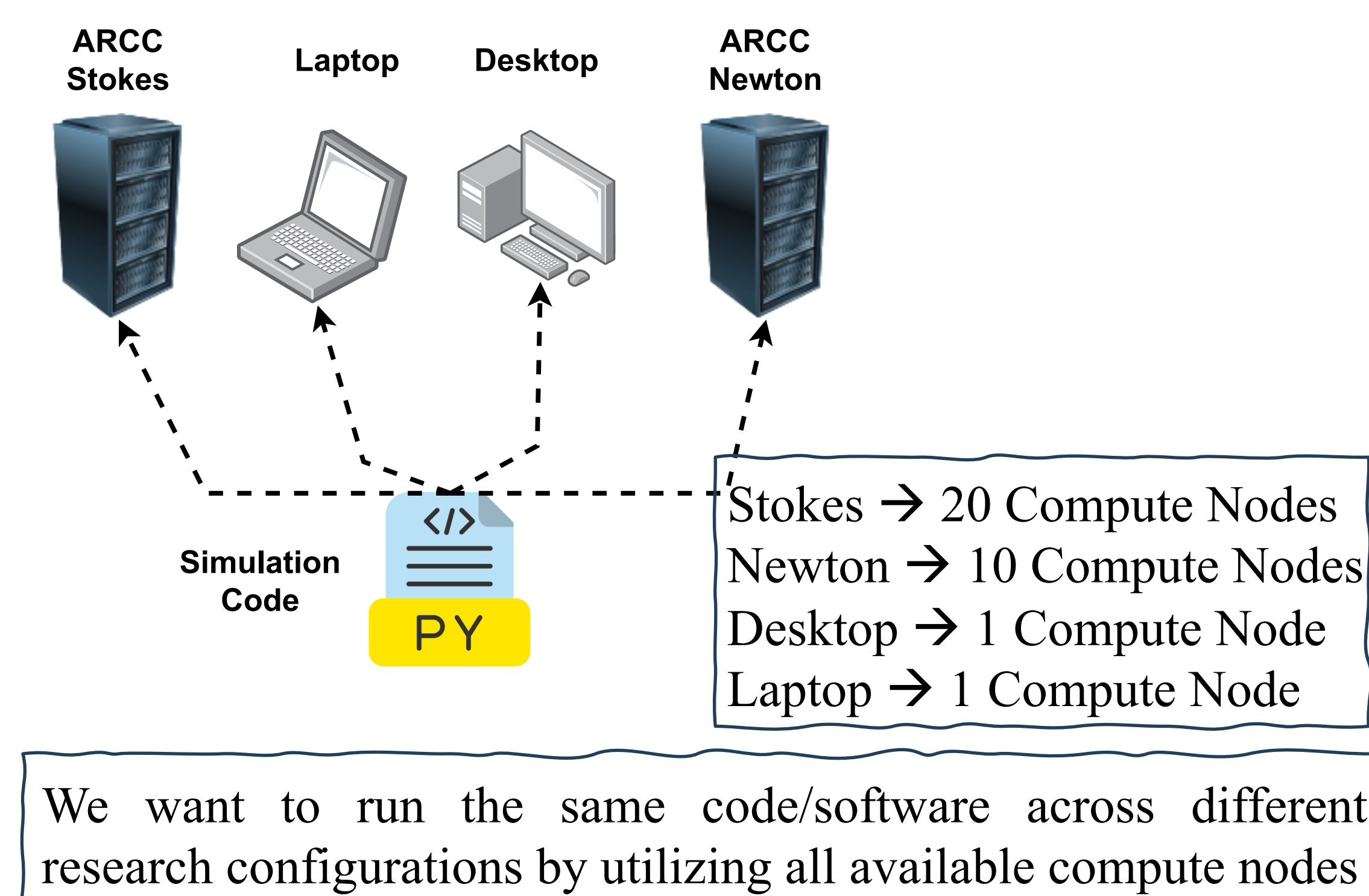
INTRODUCTION

Modern research increasingly relies on diverse computing resources ranging from personal laptops and cloud VMs to institutional HPC clusters. Managing jobs across such heterogeneous systems is often complex, requiring significant setup and expertise. To address this challenge, we present **DART**, a lightweight, platform-agnostic job distributor designed for academic research.

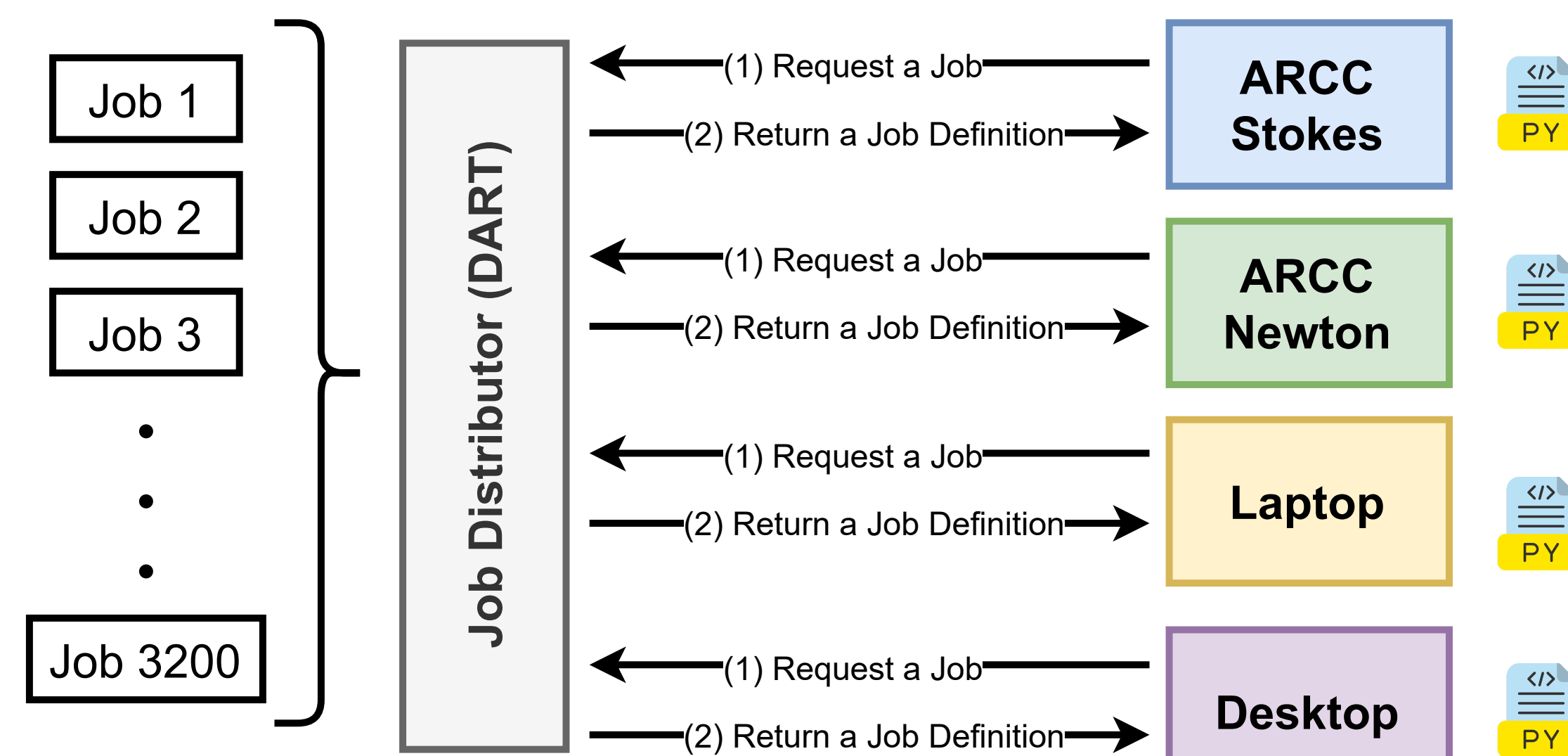
DART emphasizes **simplicity**, **portability**, and **minimal configuration**, enabling seamless job submission, monitoring, and fault-tolerant rescheduling without the overhead of traditional orchestration frameworks. Unlike tools such as **Dask** or **HTCondor**, **DART** provides an intuitive interface and quick deployment using simple **JSON** configurations, making it especially suitable for researchers handling evolving workflows.

In large-scale experiments with **37,720 jobs distributed over 68 heterogeneous machines**, DART's dynamic allocation achieved up to a **35× speedup** compared to static strategies, reducing turnaround from months to days. These results highlight **DART** as an efficient solution for managing distributed computational tasks in today's research.

CONVENTIONAL RESEARCH WORKFLOW



DART WORKFLOW



KEY CONTRIBUTIONS

Flexible Resources: Add or remove worker machines anytime during workflow.

On-Demand Scheduling: Jobs assigned when requested, not pre-allocated.

Load-Aware Distribution: Fast machines handle more jobs, slow ones handle fewer.

Failure Resilience: Detects failed machines and reassigns jobs to healthy nodes.

Reduced Tail Latency: Minimizes straggler effect → faster experiment completion.

Heterogeneous Friendly: Efficiently utilizes mixed compute environments.

ARCHITECTURE AND COMPONENTS

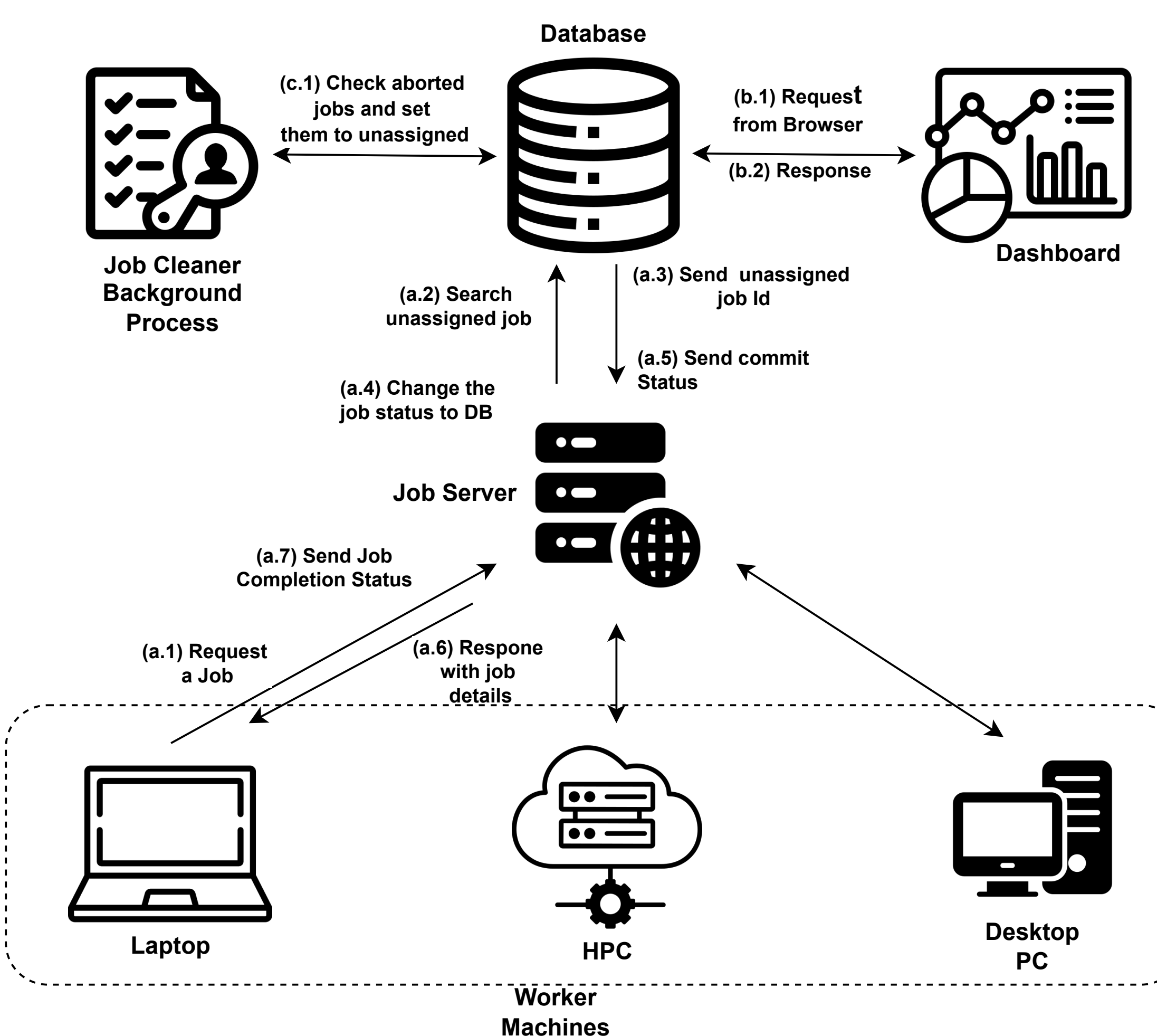


Figure: System architecture and job flow of the DART framework. Flows: (a.x) is the main job assignment and execution flow throughout the system; (b.x) is the information flow between database and dashboard; (c.x) is the handles fault tolerance of system between job cleaner and database.

JOB LIFECYCLE

A job moves from **UNASSIGNED** to **RUNNING** when a worker requests it.

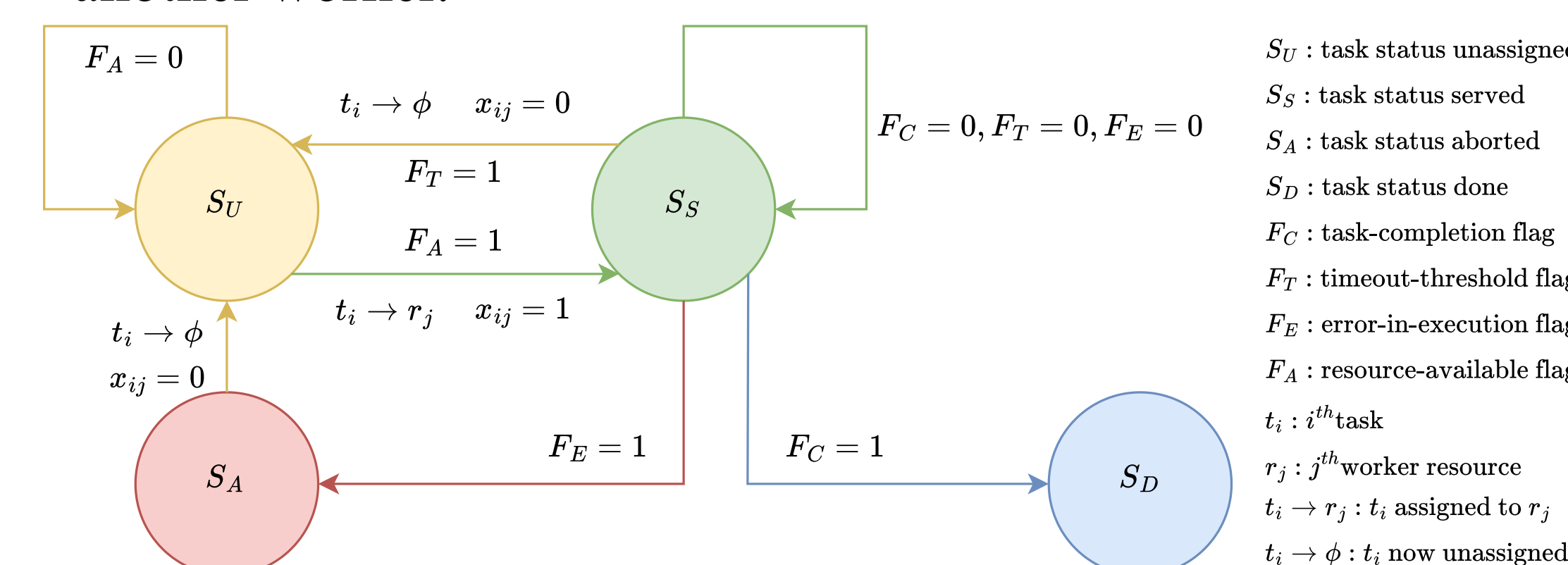
While **RUNNING**, three things can happen:

2.1) It finishes successfully → goes to **DONE**.

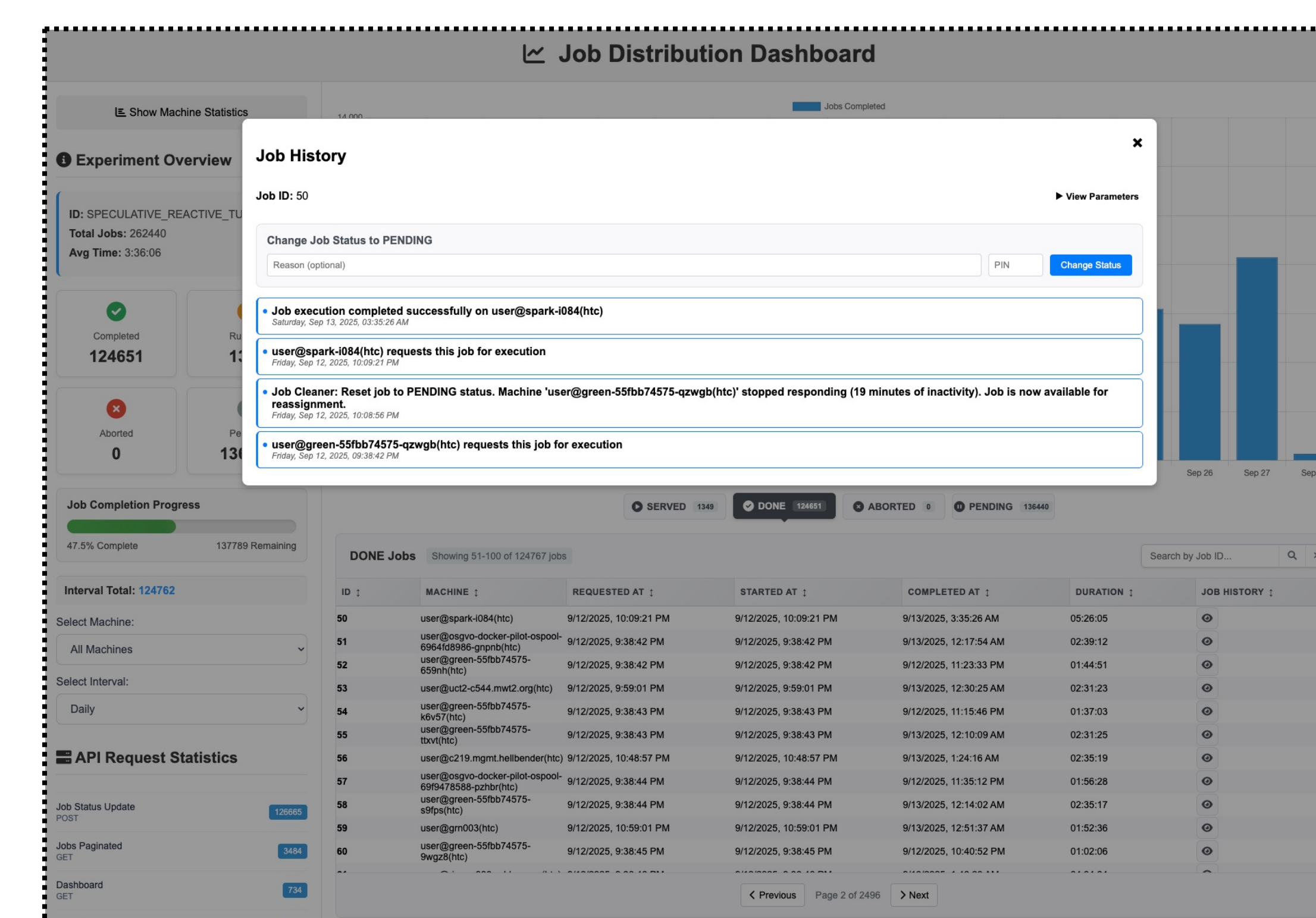
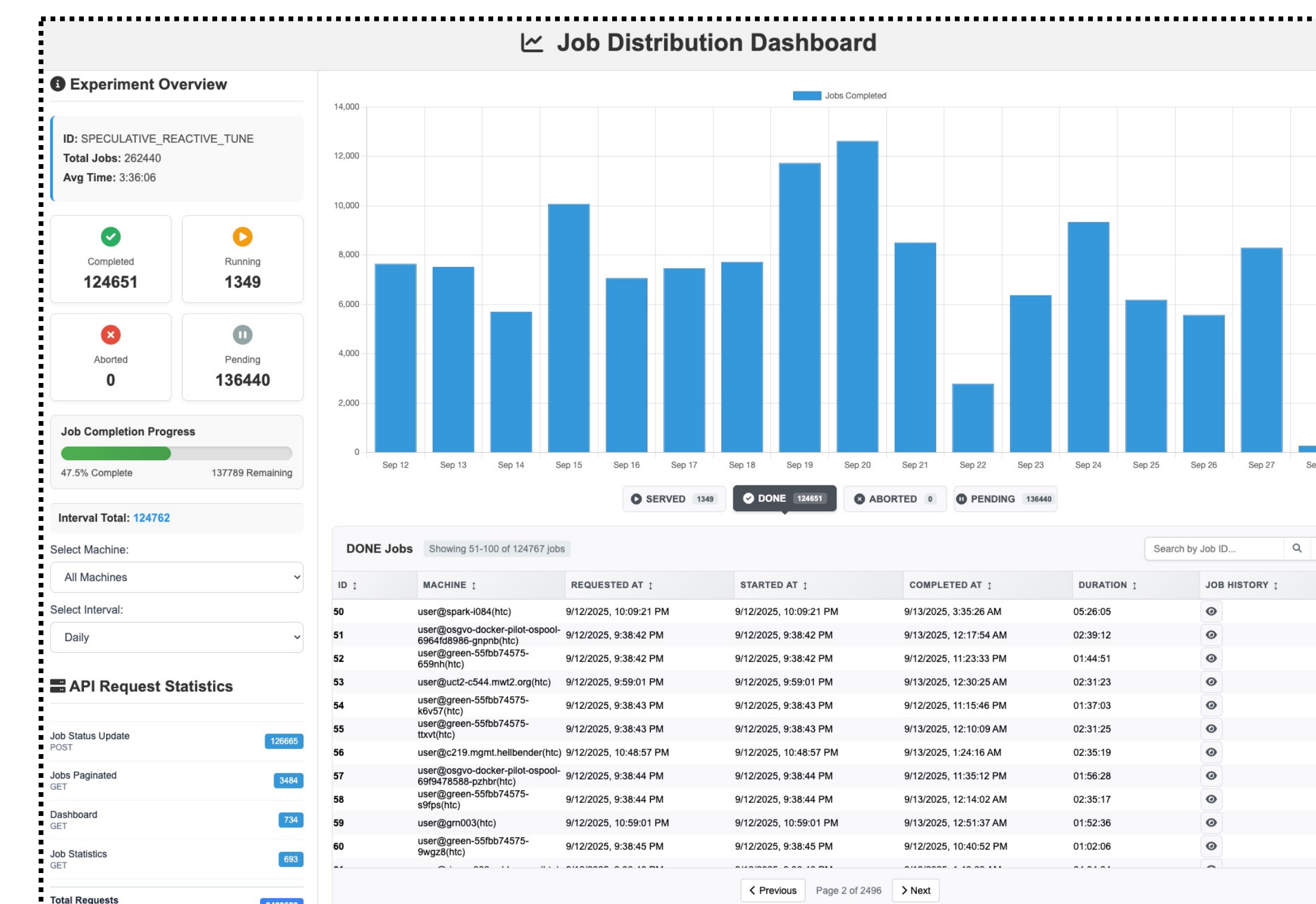
2.2) It fails (e.g., OOM or runtime error) → goes to **ABORTED**.

2.3) The worker shuts down; after a timeout → job returns to **UNASSIGNED**.

ABORTED jobs wait a short grace period, then move back to **UNASSIGNED** so the distributor can reassign them to another worker.



DART DASHBOARD



STATIC vs DYNAMIC ALLOCATION

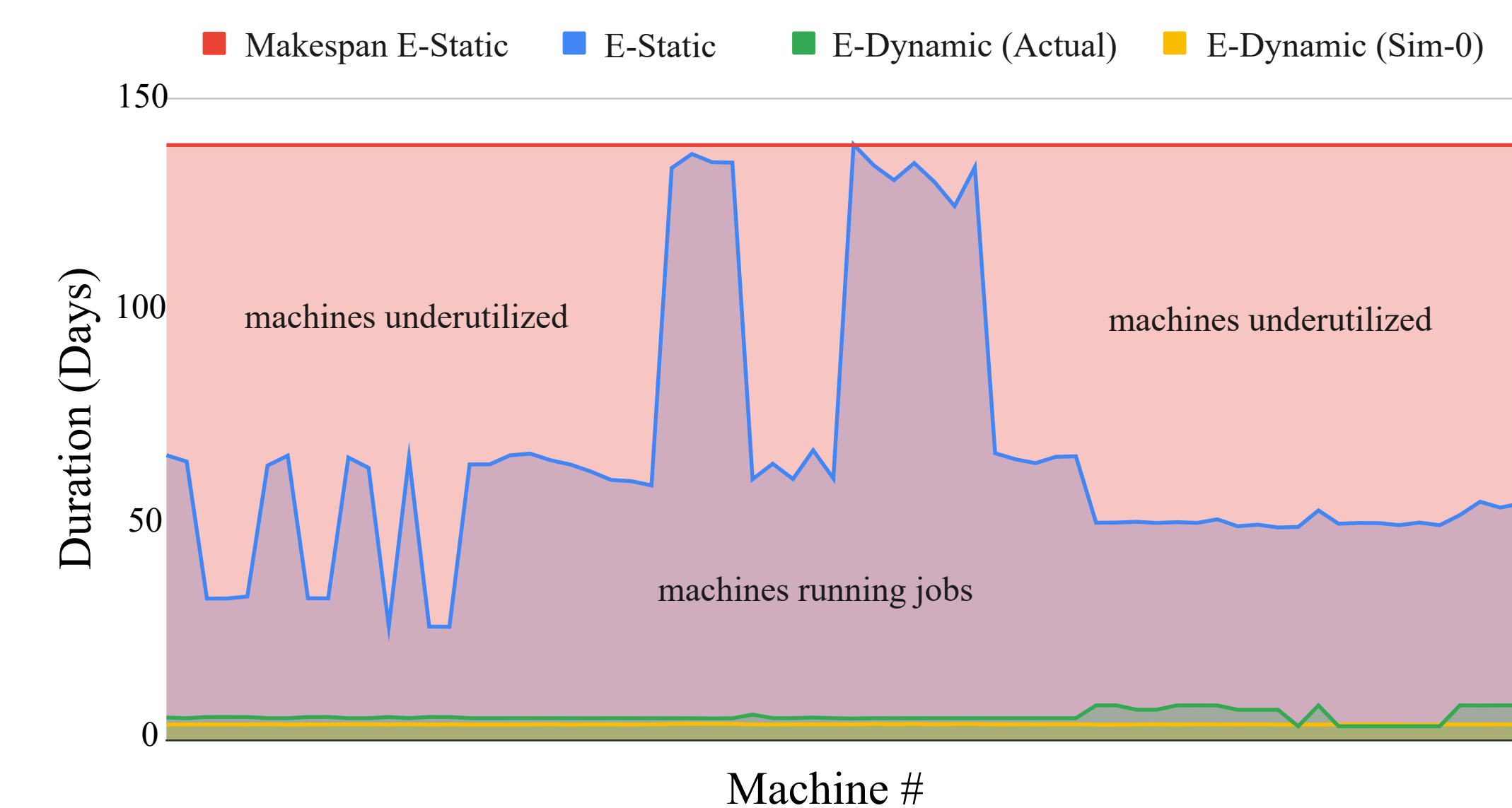


Figure: Performance comparison of static and dynamic scheduling. The figure shows the vast performance difference between the two approaches.

LEVERAGING RESOURCE HETEROGENEITY

Machines have **different speeds** — faster ones complete jobs in less time.

DART dynamically assigns more jobs to faster machines, reducing overall makespan.

Observation: Clear inverse correlation between machine runtime and number of jobs assigned.

Real-world factors (e.g., job failures, task variability, node availability) cause small deviations but overall trend is strong.

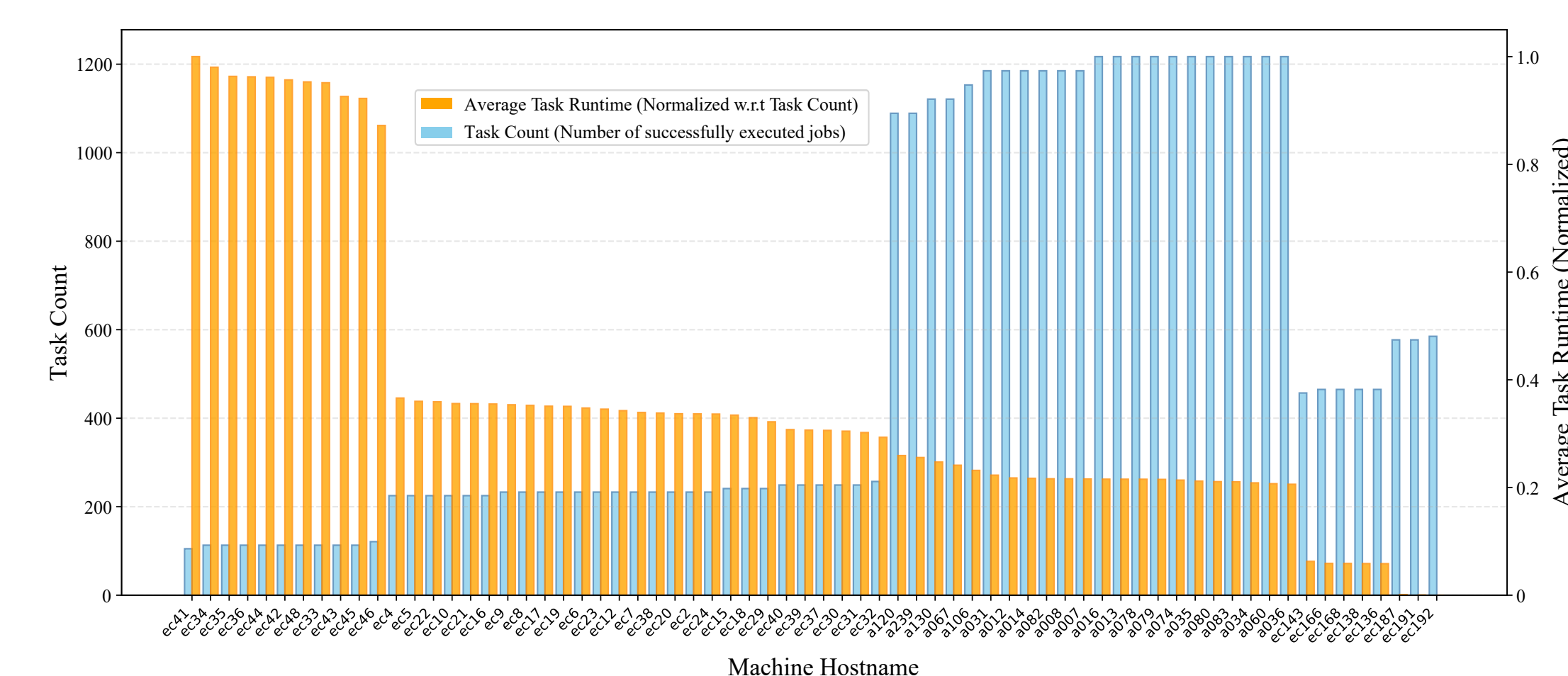


Figure: Task count vs. average task runtime, illustrating the inverse correlation between machine speed and job count and demonstrating how DART leverages resource heterogeneity.

Interested in using DART to streamline your experiments?



DART Github



DART Live Dashboard